

Unidad 9: Introducción a Git y GitHub

Git: estados, instalación y configuración

¿Qué es Git?

Git es un sistema de control de versiones gratuito y de código abierto. Fue diseñado para manejar proyectos de todos los tamaños de manera rápida y eficiente. Un sistema de control de versiones es una herramienta que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo, permitiendo que puedas recuperar versiones específicas más adelante.



Git permite a los desarrolladores gestionar el historial de versiones de su código, facilitando la colaboración en equipo y mejorando la organización del desarrollo de software. Con Git, es posible revertir el código a estados anteriores, comparar cambios, y trabajar en múltiples funcionalidades a través de ramas (branches) sin interferir en el trabajo de otros desarrolladores.

Utilidad de Git

1. **Gestión de Proyectos:** Git es extremadamente útil para proyectos de desarrollo de software de cualquier tamaño. Ya sea que trabajes solo o en un equipo grande, Git ayuda a mantener un registro detallado de todas las modificaciones realizadas en tu proyecto.
 2. **Colaboración:** Facilita la colaboración entre desarrolladores, permitiendo a varios miembros del equipo trabajar en paralelo sin conflictos. Esto se logra mediante la creación de ramas, donde cada desarrollador puede trabajar en su propia copia del código.
 3. **Control de Versiones:** Con git podemos registrar todos los cambios, lo que permite a los desarrolladores revertir a versiones anteriores del código si es necesario. Esto es particularmente útil para corregir errores o comparar diferentes versiones del proyecto.
 4. **Eficiencia y Velocidad:** Git está diseñado para ser rápido y eficiente. Las operaciones comunes son rápidas, y su arquitectura distribuida permite que cada desarrollador tenga una copia completa del historial del proyecto en su propio equipo.
-

Componentes Claves de Git

1. **Repositorio (Repository):** Es el lugar donde se almacenan todos los archivos y el historial de cambios del proyecto. Puede estar ubicado localmente en tu computadora o en un servidor remoto.
2. **Commit:** Es una instantánea del proyecto en un punto específico en el tiempo. Cada commit tiene un identificador único y contiene un mensaje que describe los cambios realizados.
3. **Rama (Branch):** Una rama es una línea de desarrollo independiente. La rama principal generalmente se llama "master" o "main", y a partir de ella se pueden crear otras ramas para desarrollar nuevas funcionalidades o corregir errores.
4. **Merge:** Es el proceso de combinar los cambios de diferentes ramas. Se utiliza para integrar el trabajo de distintas ramas en una sola.
5. **Pull Request:** Es una solicitud para revisar y fusionar los cambios de una rama en otra. Es una práctica común en proyectos colaborativos para asegurar que el código se revisa antes de integrarse en la rama principal.

Git es una herramienta esencial en el desarrollo moderno de software, proporcionando un control detallado y eficaz sobre el proceso de desarrollo, facilitando la colaboración y mejorando la calidad del código.

Instalación y Configuración de Git

Veremos cómo instalar Git en su sistema y cómo realizar la configuración inicial, que incluye establecer su nombre de usuario y correo electrónico. Tener Git correctamente instalado y configurado es crucial para cualquier proyecto de desarrollo, ya que les permitirá llevar un control preciso de sus cambios y colaborar de manera eficiente con otros desarrolladores.

Los tres formatos de GIT



Creación de un Repositorio en Git

Instrucciones paso a paso para crear un nuevo repositorio local utilizando el comando `git init`

Git es una herramienta poderosa para el control de versiones y la gestión de proyectos. Crear un repositorio local es el primer paso para empezar a usar Git en tus proyectos. A continuación, se detallan las instrucciones paso a paso para crear un nuevo repositorio local.

Paso 1: Instalar Git

Antes de comenzar, asegúrate de tener Git instalado en tu sistema. Puedes descargarlo e instalarlo desde git-scm.com.

Paso 2: Abrir la Terminal

Abre la terminal (o Git Bash en Windows) para ejecutar los comandos de Git.

Paso 3: Navegar a la Carpeta del Proyecto

Usa el comando `cd` para navegar a la carpeta donde quieres crear el repositorio. Por ejemplo, si tu proyecto está en una carpeta llamada `Proyectos\_Coder` en el directorio `Documentos`, utiliza:

...

```
cd ~/Documentos/Proyectos_Coder
```

```
'''
```

Paso 4: Inicializar el Repositorio

Una vez que estés en la carpeta del proyecto, ejecuta el siguiente comando para inicializar un nuevo repositorio de Git:

```
'''
```

```
git init
```

```
'''
```

Este comando creará una nueva subcarpeta llamada `.git` en tu directorio de proyecto. Esta carpeta contiene todos los archivos necesarios para el repositorio, como el historial de commits y la configuración del repositorio.

Paso 5: Verificar la Creación del Repositorio

Para verificar que el repositorio se ha creado correctamente, puedes ejecutar el siguiente comando:

```
'''
```

```
ls -a
```

```
'''
```

Deberías ver una lista de archivos y carpetas, incluyendo la carpeta `.git`.

Paso 6: Añadir Archivos al Repositorio

Crea o añade los archivos que quieres controlar con Git en la carpeta del proyecto. Por ejemplo, puedes crear un archivo `index.html` usando un editor de texto.

Paso 7: Añadir Archivos al Staging Area

Para añadir archivos al área de preparación (staging area), usa el comando `git add` seguido del nombre del archivo. Por ejemplo:

```
'''
```

```
git add index.html
```

```
'''
```

También puedes añadir todos los archivos en el directorio usando:

```
'''
```

```
git add .
```

```
'''
```

Paso 8: Hacer un Commit Inicial

Para guardar los cambios en el repositorio, realiza un commit inicial usando el siguiente comando:

```
...  
git commit -m "Commit inicial"
```

El mensaje entre comillas debe describir brevemente el contenido o propósito del commit.

Resumen

- **Instalar Git:** Asegúrate de tener Git instalado.
- **Abrir la Terminal:** Usa Git Bash o la terminal de tu sistema.
- **Navegar a la Carpeta del Proyecto:** Usa el comando ``cd``.
- **Inicializar el Repositorio:** Ejecuta ``git init``.
- **Verificar la Creación del Repositorio:** Usa ``ls -a`` para ver la carpeta ``.git``.
- **Añadir Archivos al Repositorio:** Usa ``git add``.
- **Hacer un Commit Inicial:** Ejecuta ``git commit -m "Commit inicial"``.

Con estos pasos, habrás creado y configurado un nuevo repositorio local en Git, listo para empezar a rastrear y gestionar los cambios en tu proyecto.

Agregar y Cometer Cambios

Descripción del uso de los comandos ``git add`` y ``git commit``

Git es una herramienta poderosa para el control de versiones, y uno de sus principales beneficios es la capacidad de gestionar los cambios en tus archivos de forma estructurada. Aquí te explicamos cómo usar los comandos ``git add`` y ``git commit`` para agregar archivos al área de preparación y confirmar los cambios en el repositorio local.

Agregar Archivos al Área de Preparación (Staging Area)

La "Staging Area" es un área temporal donde puedes preparar los cambios antes de confirmarlos en el repositorio. Este proceso te permite revisar y organizar los cambios que deseas incluir en tu próximo commit.

1. Agregar Archivos Individuales

Para agregar un archivo específico al área de preparación, utiliza el siguiente comando:

```
...
```

```
git add nombre_del_archivo
```

```
'''
```

Por ejemplo, para agregar un archivo llamado `index.html`, ejecuta:

```
'''
```

```
git add index.html
```

```
'''
```

2. Agregar Todos los Archivos Nuevos y Modificados

Si deseas agregar todos los archivos nuevos y modificados en el directorio, usa el siguiente comando:

```
'''
```

```
git add .
```

```
'''
```

Este comando agrega todos los cambios en el directorio actual al área de preparación.

3. Verificar el Estado del Repositorio

Para ver qué archivos han sido añadidos al área de preparación, puedes usar:

```
'''
```

```
git status
```

```
'''
```

Este comando muestra una lista de archivos que están en el área de preparación y los que aún no lo están.

Confirmar los Cambios en el Repositorio (Commit)

Una vez que los archivos están en el área de preparación, el siguiente paso es confirmar los cambios en el repositorio local con un commit. Un commit es una instantánea del proyecto en un momento específico, y cada commit debe tener un mensaje descriptivo que explique los cambios realizados.

1. Hacer un Commit

Para confirmar los cambios en el repositorio, usa el siguiente comando:

```
'''
```

```
git commit -m "Mensaje descriptivo de los cambios"
```

'''

El mensaje entre comillas debe describir brevemente los cambios realizados. Por ejemplo:

'''

```
git commit -m "Añadir el archivo index.html con la estructura básica del proyecto"
```

'''

2. Verificar el Historial de Commits

Para revisar el historial de commits en el repositorio, puedes usar:

'''

```
git log
```

'''

Este comando muestra una lista de todos los commits realizados, incluyendo el mensaje, el autor, y la fecha de cada commit.

Resumen

- **Agregar Archivos al Área de Preparación:**

- ``git add nombre_del_archivo`` para agregar archivos específicos.
- ``git add .`` para agregar todos los archivos nuevos y modificados.
- ``git status`` para verificar el estado del repositorio.

- **Confirmar los Cambios en el Repositorio:**

- ``git commit -m "Mensaje descriptivo de los cambios"`` para hacer un commit.
- ``git log`` para revisar el historial de commits.

Estos pasos te permiten gestionar de manera efectiva los cambios en tu proyecto, asegurando que cada modificación esté registrada y documentada correctamente en tu repositorio de Git.

Trabajo con Ramas en Git

Explicación sobre la creación y manejo de ramas en Git

Git permite a los desarrolladores trabajar en diferentes funcionalidades y correcciones de errores de forma aislada mediante el uso de ramas. Las ramas son versiones paralelas del proyecto que pueden ser modificadas independientemente antes de ser fusionadas con la rama principal. A continuación, se explican los conceptos clave y los comandos para crear y manejar ramas en Git.

Creación de Ramas

1. Crear una Nueva Rama

Para crear una nueva rama, utiliza el comando `git branch` seguido del nombre de la nueva rama:

```
...  
git branch nombre_de_la_rama  
...
```

Por ejemplo, para crear una rama llamada `nueva_funcionalidad`, usa:

```
...  
git branch nueva_funcionalidad  
...
```

2. Listar Ramas Existentes

Para ver una lista de todas las ramas en el repositorio, usa:

```
...  
git branch  
...
```

La rama actual estará marcada con un asterisco (*).

Cambio entre Ramas

1. Cambiar a una Rama Existente

Para cambiar a una rama existente, utiliza el comando `git checkout` seguido del nombre de la rama:

```
...  
git checkout nombre_de_la_rama  
...
```


Por ejemplo, para cambiar a la rama `nueva\_funcionalidad`, usa:

```
...  
git checkout nueva_funcionalidad  
...
```

2. Crear y Cambiar a una Nueva Rama en un Solo Paso

Puedes crear una nueva rama y cambiar a ella en un solo comando usando la opción `b`:

```
...  
git checkout -b nombre_de_la_rama  
...
```

Por ejemplo, para crear y cambiar a una nueva rama llamada `nueva\_funcionalidad`, usa:

```
...  
git checkout -b nueva_funcionalidad  
...
```

Fusión de Ramas

Fusionar una Rama con la Rama Actual:

Para fusionar los cambios de una rama en la rama actual, primero asegúrate de estar en la rama a la que deseas fusionar, generalmente la rama principal (`master` o `main`), y luego usa el comando `git merge` seguido del nombre de la rama que deseas fusionar:

Por ejemplo, para fusionar la rama `nueva\_funcionalidad` en la rama `master`, usa:

```
...  
  
git checkout master  
git merge nombre_de_la_rama  
  
...  
  
...  
  
git checkout master  
git merge nueva_funcionalidad  
  
...
```

Resumen

- **Crear Ramas:**
 - ``git branch nombre_de_la_rama`` para crear una nueva rama.
 - ``git branch`` para listar las ramas existentes.
- **Cambiar entre Ramas:**
 - ``git checkout nombre_de_la_rama`` para cambiar a una rama existente.
 - ``git checkout -b nombre_de_la_rama`` para crear y cambiar a una nueva rama en un solo paso.
- **Fusionar Ramas:**
 - ``git merge nombre_de_la_rama`` para fusionar los cambios de una rama en la rama actual.

El trabajo con ramas en Git permite una gestión eficiente del desarrollo de software, facilitando la colaboración y la experimentación sin afectar la estabilidad de la rama principal del proyecto.

Introducción a GitHub

¿Qué es GitHub?

Definición de GitHub como una plataforma para alojar repositorios Git

GitHub es una plataforma de alojamiento de repositorios Git en la nube. Esta herramienta no solo permite almacenar y gestionar el código fuente de proyectos de software, sino que también facilita la colaboración entre desarrolladores y equipos de desarrollo. A continuación, se explican sus principales características y usos.



Definición y Funcionalidad

1. Plataforma de Alojamiento de Repositorios Git

GitHub proporciona un espacio centralizado donde los desarrolladores pueden alojar sus repositorios Git. Esto significa que cualquier proyecto que utilice Git para el control de versiones puede ser almacenado y gestionado en GitHub. Además, GitHub ofrece una interfaz web fácil de usar para navegar por los repositorios, ver el historial de commits, y gestionar el proyecto.

2. Colaboración y Trabajo en Equipo

Una de las principales ventajas de GitHub es su capacidad para facilitar la colaboración. Los desarrolladores pueden trabajar juntos en proyectos compartidos, hacer revisiones de código, y discutir cambios a través de comentarios y pull requests. GitHub también permite a los equipos gestionar el flujo de trabajo mediante la asignación de tareas y el seguimiento de problemas (issues).

Usos y Beneficios

1. Compartir Proyectos

GitHub permite a los desarrolladores compartir sus proyectos con otros. Esto es particularmente útil para proyectos de código abierto, donde cualquier persona puede contribuir. Los repositorios públicos pueden ser clonados y modificados por cualquier usuario, promoviendo la colaboración y el intercambio de conocimientos.

2. Gestión de Versiones y Historial de Cambios

Al igual que Git, GitHub permite a los desarrolladores llevar un registro detallado de todos los cambios realizados en el código. Esto facilita la revisión de versiones anteriores y la recuperación de estados anteriores del proyecto en caso de errores.

3. Revisiones y Comentarios

GitHub proporciona herramientas para la revisión de código, donde los desarrolladores pueden comentar sobre líneas específicas de código y discutir posibles mejoras o correcciones. Esta funcionalidad es crucial para mantener la calidad del código y asegurar que todos los cambios sean revisados antes de ser fusionados en la rama principal del proyecto.

4. Automatización y CI/CD

GitHub se integra con una variedad de herramientas de automatización y de integración continua/entrega continua (CI/CD). Esto permite a los desarrolladores automatizar pruebas, despliegues y otras tareas, mejorando la eficiencia y la confiabilidad del desarrollo del software.

Resumen

- **Plataforma de Alojamiento de Repositorios Git:** GitHub proporciona un espacio centralizado para almacenar y gestionar proyectos utilizando Git.
- **Colaboración y Trabajo en Equipo:** Facilita la colaboración entre desarrolladores, permitiendo revisiones de código y discusiones sobre cambios.
- **Compartir Proyectos:** Ideal para proyectos de código abierto y la colaboración global.
- **Gestión de Versiones e Historial de Cambios:** Permite llevar un registro detallado de todos los cambios realizados en el código.
- **Revisiones y Comentarios:** Herramientas para la revisión de código y discusión de mejoras.
- **Automatización y CI/CD:** Integración con herramientas de automatización para mejorar la eficiencia del desarrollo del software.

GitHub es una herramienta esencial en el desarrollo moderno de software, proporcionando una plataforma robusta para la gestión de proyectos, la colaboración y la automatización de tareas.

Django: introducción, MVC y creación de un proyecto

¿Qué es Django?

Introducción a Django como un framework de desarrollo web en Python

Django es un framework de desarrollo web de alto nivel que está escrito en Python y promueve el desarrollo rápido y un diseño limpio y pragmático. Este framework es gratuito y de código abierto, lo que lo hace accesible para desarrolladores de todo el mundo. A continuación, se explica qué es Django, sus características principales y sus ventajas para la creación de proyectos web complejos.



Definición y Funcionalidad

1. Framework de Desarrollo Web

Django proporciona una estructura sólida para el desarrollo de aplicaciones web, permitiendo a los desarrolladores centrarse en escribir su aplicación sin tener que reinventar la rueda. Incluye una gran cantidad de funcionalidades integradas que facilitan el desarrollo web, como un sistema de administración, formularios, autenticación, y más.

2. Basado en Python

Al estar escrito en Python, Django hereda la simplicidad y legibilidad de este lenguaje. Python es conocido por su sintaxis clara y concisa, lo que hace que el código de Django sea fácil de leer y mantener.

Simplicidad y Eficiencia

1. Desarrollo Rápido

Django está diseñado para ayudar a los desarrolladores a llevar sus aplicaciones del concepto a la realidad lo más rápido posible. Esto es posible gracias a su enfoque en la reutilización de componentes y a la automatización de tareas repetitivas.

2. Administración Automática

Una de las características más destacadas de Django es su interfaz de administración automática. Al definir los modelos de datos, Django genera automáticamente una interfaz de administración web, lo que permite a los desarrolladores gestionar el contenido de su sitio de manera sencilla.

Capacidad para Proyectos Web Complejos

1. Escalabilidad

Django es adecuado tanto para pequeños proyectos como para aplicaciones web de gran escala. Su arquitectura basada en componentes permite a los desarrolladores construir aplicaciones escalables y mantenibles.

2. Seguridad

La seguridad es una prioridad en Django. El framework incluye protección contra muchas vulnerabilidades de seguridad comunes, como la inyección SQL, los ataques de scripts entre sitios (XSS), y la falsificación de solicitudes entre sitios (CSRF).

3. Comunidad y Documentación

Django cuenta con una comunidad activa y una documentación extensa y bien organizada. Esto facilita el aprendizaje y la resolución de problemas, ya que los desarrolladores pueden encontrar fácilmente recursos y ejemplos en línea.

Resumen

- **Framework de Desarrollo Web:** Django proporciona una estructura sólida y un conjunto de herramientas para el desarrollo de aplicaciones web.
- **Basado en Python:** Hereda la simplicidad y legibilidad de Python, lo que facilita el desarrollo y mantenimiento del código.

- **Desarrollo Rápido:** Permite llevar aplicaciones del concepto a la realidad rápidamente mediante la reutilización de componentes y la automatización de tareas repetitivas
- **Administración Automática:** Genera una interfaz de administración web para la gestión del contenido.
- **Escalabilidad:** Adecuado para proyectos de cualquier tamaño, desde pequeños sitios web hasta grandes aplicaciones.
- **Seguridad:** Incluye protección contra muchas vulnerabilidades de seguridad comunes.
- **Comunidad y Documentación:** Cuenta con una comunidad activa y una extensa documentación que facilita el aprendizaje y la resolución de problemas.

Django es una herramienta poderosa para el desarrollo web, proporcionando simplicidad y eficiencia sin sacrificar la capacidad de manejar proyectos complejos.

Django: Primer View, creación y uso de templates

Creación de la Primera Vista en Django

Pasos para crear una nueva vista en Django

Crear una vista en Django es uno de los primeros pasos para desarrollar una aplicación web con este framework. A continuación, se describen los pasos necesarios para crear una vista, incluyendo la creación de un archivo `views.py`, la definición de una función de vista y la configuración de las URLs.

Paso 1: Crear el Archivo

1. Ubicar el Archivo [views.py](#):

En tu proyecto Django, navega a la carpeta de la aplicación en la que deseas crear la vista. Dentro de esta carpeta, crea un archivo llamado `views.py` si no existe ya.

2. Importar las Bibliotecas Necesarias:

Abre `views.py` y añade las importaciones necesarias:

```
'''python
from django.http import HttpResponse
'''
```

Paso 2: Definir una Función de Vista

1. Crear una Función de Vista:

Dentro del archivo `views.py`, define una función que manejará la lógica de la vista. Esta función debe aceptar al menos un parámetro, que es el objeto `request`, y devolver un objeto `HttpResponse`. Por ejemplo:

```
```python
def saludo(request):
 return HttpResponse("Hola Django - Coder")
...
```
```

Paso 3: Configurar las URLs

1. Modificar el Archivo [urls.py](#):

Navega al archivo `urls.py` de tu aplicación (o del proyecto si estás configurando las URLs a nivel global). Este archivo se encuentra típicamente en la carpeta de tu proyecto o aplicación.

2. Importar la Vista:

Importa la función de vista que acabas de crear en `views.py`. Por ejemplo:

```
```python
from .views import saludo
...
```
```

3. Configurar la URL:

Añade una entrada en el archivo `urls.py` para asociar una URL específica con la función de vista. Por ejemplo:

```
```python
from django.urls import path

urlpatterns = [
 path('saludo/', saludo),
]
...
```
```

Resumen

- **Crear el Archivo [views.py](#):**
 - Navega a la carpeta de tu aplicación y crea el archivo `views.py` si no existe.
 - Importa `HttpResponse` desde `django.http`.
- **Definir una Función de Vista:**
 - Dentro de `views.py`, define una función que acepte un parámetro `request` y devuelva un `HttpResponse`.
- **Ejemplo de función de vista:**

```
```python
def saludo(request):
 return HttpResponse("Hola Django - Coder")

...`
```

- **Configurar las URLs:**
  - Navega al archivo `urls.py` de tu aplicación o proyecto.
  - Importa la función de vista desde `views.py`.
  - Añade una entrada en `urlpatterns` para asociar la URL con la función de vista.
- Ejemplo de configuración de URL:

```
```python
from django.urls import path
from .views import saludo

urlpatterns = [
    path('saludo/', saludo),
]

...`
```

Estos pasos permiten configurar y visualizar tu primera vista en Django, estableciendo las bases para el desarrollo de aplicaciones web más complejas.

Uso de Templates en Django

Explicación sobre cómo utilizar templates en Django para separar la lógica de la vista de la presentación

Django facilita la creación de aplicaciones web utilizando el patrón Modelo-Vista-Controlador (MVC), donde los templates juegan un papel crucial al permitir la separación de la lógica del backend y la presentación del frontend. A continuación, se describen los pasos para crear y renderizar un template en Django.

Paso 1: Configuración del Sistema de Templates

1. Configurar la Carpeta de Templates:

Asegúrate de que la carpeta de templates esté correctamente configurada en tu proyecto Django. En el archivo `settings.py`, define el directorio de templates en la sección `TEMPLATES`:

```
'''python
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'templates')],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

'''
```

Paso 2: Crear un Template

1. Crear la Carpeta de Templates:

Dentro de tu aplicación, crea una carpeta llamada `templates`. Dentro de esta carpeta, puedes organizar tus templates en subcarpetas. Por ejemplo, para una aplicación llamada `miapp`, crea una carpeta `miapp` dentro de `templates`:

```
'''
```

```
miapp/  
  templates/  
    miapp/  
      saludo.html
```

```
...
```

2. Crear el Archivo HTML del Template:

Crea un archivo HTML en la carpeta correspondiente. Por ejemplo, `saludo.html`:

```
```html  
<!DOCTYPE html>
<html>
<head>
 <title>Saludo</title>
</head>
<body>
 <h1>{{ mensaje }}</h1>
</body>
</html>
```

```
...
```

---

## Paso 3: Renderizar el Template

### 1. Modificar la Vista para Renderizar el Template:

Abre el archivo `views.py` de tu aplicación y modifica la función de vista para renderizar el template utilizando el método `render`:

```
```python  
from django.shortcuts import render  
  
def saludo(request):  
    contexto = {'mensaje': 'Hola Django - Coder'}  
    return render(request, 'miapp/saludo.html', contexto)
```

```
...
```

2. Configurar la URL para la Vista:

Asegúrate de que la URL correspondiente esté configurada en `urls.py`:

```
```python  
from django.urls import path
from .views import saludo
```

```
urlpatterns = [
 path('saludo/', saludo),
]
```

```
...
```

---

## Resumen

- **Configurar la Carpeta de Templates:**

- En `settings.py`, define la ubicación de los templates en `TEMPLATES`.

- **Crear un Template:**

- Dentro de tu aplicación, crea una carpeta `templates` y dentro de ella una subcarpeta con el nombre de tu aplicación.

- Crea un archivo HTML en esta carpeta. Ejemplo: `saludo.html`.

- **Renderizar el Template:**

- En `views.py`, usa el método `render` para devolver el template junto con el contexto.

- Ejemplo de función de vista:

```
```python  
from django.shortcuts import render  
  
def saludo(request):  
    contexto = {'mensaje': 'Hola Django - Coder'}  
    return render(request, 'miapp/saludo.html', contexto)  
  
...
```

- **Configurar la URL para la Vista:**

- En `urls.py`, añade una ruta para la vista.

- Ejemplo de configuración de URL:

```
```python  
from django.urls import path
from .views import saludo

urlpatterns = [
 path('saludo/', saludo),
]

...
```

Usar templates en Django permite separar la lógica del backend de la presentación del frontend, haciendo que el desarrollo de aplicaciones web sea más organizado y eficiente.

---

## Entornos Virtuales y Paquetes en Python

Un entorno virtual en Python es una herramienta que proporciona a los desarrolladores un espacio aislado para instalar paquetes y librerías necesarios para un proyecto específico, sin afectar el resto del sistema. Esto es especialmente útil para evitar conflictos entre diferentes proyectos que pueden requerir distintas versiones de los mismos paquetes.

### Importancia de los Entornos Virtuales

#### 1. Aislamiento de Dependencias

Los entornos virtuales permiten aislar las dependencias de cada proyecto. Esto significa que cada proyecto puede tener sus propias versiones de las librerías, lo que evita conflictos y problemas de compatibilidad.

#### 2. Facilidad de Gestión

Usar entornos virtuales facilita la gestión de las dependencias. Puedes instalar, actualizar o eliminar paquetes sin afectar otros proyectos.

#### 3. Reproducibilidad

Los entornos virtuales ayudan a asegurar que el entorno de desarrollo sea el mismo en diferentes máquinas, lo que facilita la colaboración y el despliegue del proyecto.

---

## Instrucciones para Crear y Activar un Entorno Virtual con virtualenv (hay otras opciones)

### Paso 1: Instalar la Librería virtualenv

#### 1. Instalar virtualenv

Antes de crear un entorno virtual, necesitas instalar `virtualenv` si no lo tienes ya instalado. Puedes hacerlo usando `pip`:

```
...
pip install virtualenv
...
```

## Paso 2: Crear un Entorno Virtual

### 1. Crear la Carpeta del Proyecto

Navega a la carpeta de tu proyecto o crea una nueva carpeta donde desees que se aloje tu proyecto. Por ejemplo:

```
'''
mkdir mi_proyecto
cd mi_proyecto
'''
```

### 2. Crear el Entorno Virtual

Dentro de la carpeta del proyecto, crea un entorno virtual ejecutando el siguiente comando:

```
'''
virtualenv venv
'''
```

Aquí, `venv` es el nombre del entorno virtual. Puedes nombrarlo como prefieras.

## Paso 3: Activar el Entorno Virtual

### 1. Activar el Entorno Virtual en Windows

Para activar el entorno virtual en Windows, usa el siguiente comando:

```
'''
venv\Scripts\activate
'''
```

### 2. Activar el Entorno Virtual en MacOS/Linux

Para activar el entorno virtual en MacOS o Linux, usa el siguiente comando:

```
'''
source venv/bin/activate
'''
```

### 3. Verificación

Una vez activado, deberías ver el nombre del entorno virtual (por ejemplo, `(venv)`) al principio de la línea de comandos, indicando que el entorno virtual está activo.

## Paso 4: Instalar Paquetes en el Entorno Virtual

### 1. Instalar Paquetes

Con el entorno virtual activo, puedes instalar los paquetes necesarios utilizando `pip`. Por ejemplo:

```
'''
pip install django
'''
```

### 2. Guardar Dependencias

Para guardar una lista de las dependencias instaladas, usa:

```
'''
pip freeze > requirements.txt
'''
```

### 3. Instalar Dependencias desde un Archivo

Para instalar todas las dependencias listadas en un archivo `requirements.txt`, usa:

```
'''
pip install -r requirements.txt
'''
```

## Paso 5: Desactivar el Entorno Virtual

### 1. Desactivar el Entorno Virtual

Para desactivar el entorno virtual cuando hayas terminado, simplemente ejecuta:

```
'''
deactivate
'''
```

---

## Resumen

- **Aislamiento de Dependencias:** Evita conflictos entre proyectos.
- **Facilidad de Gestión:** Simplifica la instalación y actualización de paquetes.

- **Reproducibilidad:** Asegura que el entorno de desarrollo sea consistente en diferentes máquinas.

## Instrucciones:

- **Instalar virtualenv:**

```
'''
```

```
pip install virtualenv
```

```
'''
```

- **Crear un Entorno Virtual:**

```
'''
```

```
virtualenv venv
```

```
'''
```

- **Activar el Entorno Virtual:**

- Windows: `venv\Scripts\activate`

- MacOS/Linux: `source venv/bin/activate`

- **Instalar Paquetes:**

```
'''
```

```
pip install <paquete>
```

```
'''
```

- **Guardar Dependencias:**

```
'''
```

```
pip freeze > requirements.txt
```

```
'''
```

- **Desactivar el Entorno Virtual:**

```
'''
```

```
deactivate
```

```
'''
```

Utilizar entornos virtuales en Python es una práctica esencial para gestionar las dependencias de manera efectiva y mantener un entorno de desarrollo ordenado y libre de conflictos.



---

# Material complementario:

## Recursos complementarios

### ¿Quieres saber más?

- [Git Cheat Sheet](#) | GitHub Education
- [Tutoriales de uso GitHub](#) | GitHub
- [\[Cómo usar la integración Git en Visual Studio Code](#) | Digital Ocean

### Django

- [Tutorial Django básico \(Python\)](#) | Developer

### Entornos virtuales

- [Creación de entornos virtuales.](#)
- [Entornos virtuales y paquetes](#)

[Tutorial instalación Git \(Windows - Mac](#)

[Tutorial Instalación Django](#)